

Yazılım Tasarımı: Boyut Sınırıyla Yönlendirilen Modüler Ayrıştırma

ÖFY

October 2026

Özet

Bir kod tabanında her dosyayı belirli bir satır sayısının altında tutma alışkanlığı, **boyut sınırıyla yönlendirilen modüler ayrıştırma** olarak adlandırılabilir. Uygulamanın iki bileşeni vardır: büyük bir parçayı küçük parçalara bölme işlemi olan ayrıştırma ve bölmenin ne zaman yapılacağını belirleyen boyut sınırı. Sınır, bir tasarım ilkesi değil bir tetikleyicidir; aşıldığında dosyanın bölünmesi gerektiğini bildirir. Uygulamanın kararlılık üretmesi üç mekanizmaya dayanır: dosya tek seferde akılda tutulabilecek boyutta kalır, küçük kalmak zorunda olan dosya tek bir işe daralır ve bir değişikliğin etkisi küçük parçanın içinde kalır. Satır sayısı bu üç özelliğin kendisi değil, onların ucuz ölçülebilen bir vekilidir. Bu nedenle sınır, tek iş yapan bir dosyayı zorla bölmek için kullanıldığında fayda yerine zarar üretir.

Giriş

Bir yazılım projesi büyüdükçe dosyalar da büyür. Belirli bir noktadan sonra bir dosyada yapılan değişiklik, aynı dosyanın ilgisiz görünen başka bir yerini bozmaya başlar. Bu duruma karşı sık başvuru alan pratik bir önlem, her dosyaya bir üst satır sınırı koymaktır: örneğin 200 satır. Dosya bu sınırı aştığında ikiye ya da daha fazla parçaya bölünür.

Bu önlem çoğunlukla bir alışkanlık olarak uygulanır ve adı konmaz. Adının konması iki nedenle gereklidir. Birincisi, uygulamanın neden işe yaradığı ancak arkasındaki kavramlar ayrıştırıldığında görülür. İkincisi, aynı kavramlar bilindiğinde sınırın ne zaman esnetilebileceği de bilinir.

Yaygın kabul, satır sınırının kendisinin bir kalite ölçütü olduğudur. Bu kabul eksiktir. Sınır yalnızca bir işarettir; kaliteyi üreten şey, sınırın dolaylı olarak zorladığı yapısal özelliklerdir.

Not, uygulamanın adlandırılması ve kararlılık üretme mekanizması ile sınırlıdır. Sınırın sayısal değerinin nasıl seçileceği ve farklı programlama dillerindeki karşılıkları kapsam dışındadır.

Uygulamanın Adı

Ayrıştırma ve modülerleştirme

Ayrıştırma (*decomposition*), büyük bir parçanın, her biri tek başına anlaşılabilen küçük parçalara bölünmesidir. Aynı işlemin daha geniş adı **modülerleştirme**dir (*modularization*): sistem, aralarındaki bağlantılar sınırlı tutulan bağımsız birimlere, yani modüllere ayrılır.

Ayrıştırmanın nasıl yapılacağı sorusu yazılım mühendisliğinin en eski sorularından biridir. Parnas (1972), bir sistemin hangi ölçüte göre modüllere bölüneceğinin, sistemin sonradan ne kadar kolay değiştirilebileceğini belirlediğini göstermiştir. Simon (1962) ise aynı yapıyı genel sistemler için tarif etmiş ve parçaları kendi içinde sıkı, birbirleriyle gevşek bağlı sistemleri *neredeyse ayrıştırılabilir* (*nearly decomposable*) olarak adlandırmıştır.

Boyut sınırı

Satır kuralının kendisi bir **boyut sınırıdır** (*size constraint*). Her dosyaya bir üst sınır konur ve sınır aşıldığında dosya bölünür. Aynı kural gündelik kullanımda *karmaşıklık bütçesi* (*complexity budget*) olarak da anılır; bu ifade yerleşik bir literatür terimi değil, bir benzetmedir. Benzetmenin içeriği şudur: her dosyanın harcayabileceği sınırlı bir karmaşıklık payı vardır ve pay tükendiğinde yeni bir dosya açılır.

Bileşik terim

İki bileşen birlikte ele alındığında uygulamanın adı **boyut sınırıyla yönlendirilen modüler ayrıştırma** olur. Terimdeki iş bölümü şöyledir:

- Ayrıştırma, yapılan işlemi adlandırır.
- Boyut sınırı, işlemin ne zaman yapılacağını belirler.

Sınır, bölmenin nereden yapılacağını söylemez. Bu karar ayrıca verilir ve çoğunlukla dosyanın içindeki farklı işlerin sınırı boyunca yapılır.

Kararlılığın Üç Mekanizması

Bilişsel yük sınırı

Bilişsel yük (*cognitive load*), bir işi yaparken çalışma belleğinde aynı anda tutulması gereken bilgi miktarıdır (Sweller, 1988). Çalışma belleğinin kapasitesi dardır; Miller (1956) bu kapasiteyi yaklaşık yedi öge olarak tarif etmiştir.

Bir dosyada değişiklik yapan kişi, dosyadaki değişkenleri, fonksiyonları ve bunların birbirine etkisini akılda tutmak zorundadır. Dosya küçükse bu bilginin tamamı tek seferde akılda tutulabilir. Dosya büyükse bir kısmı unutulur ve hata unutilan kısımdan çıkar. Örneğin 800 satırlık bir dosyanın başında tanımlanan bir değişkenin, dosyanın sonundaki bir fonksiyonda da kullanıldığı gözden kaçabilir; 200 satırlık bir dosyada aynı bağlantı ekranda görünür durumdadır.

Yüksek uyum

Uyum (*cohesion*), bir modülün içindeki ögelerin aynı işe ne ölçüde hizmet ettiğidir. Kavram, yapısal tasarım çalışmalarında tanımlanmıştır (Stevens, Myers ve Constantine, 1974). Yüksek uyumlu bir dosyada her satır aynı amaca yöneliktir.

Boyut sınırı uyumu dolaylı olarak zorlar. Küçük kalmak zorunda olan bir dosyaya ikinci bir iş sığmaz; ikinci iş ayrı bir dosyaya gitmek zorunda kalır. Bu sonuç, **tek sorumluluk ilkesine** (*Single Responsibility Principle*, SRP) denk gelir. İlkeye göre bir modülün değişmesi için yalnızca tek bir neden bulunmalıdır (Martin, 2003). Örneğin hem veritabanı sorgusunu hem de bu sorgunun sonucunu ekrana biçimlendiren kodu içeren bir dosyanın iki ayrı değişme nedeni vardır; sınır aşıldığında bu iki iş doğal olarak iki dosyaya ayrılır.

Değişikliğin yerelleşmesi

Değişikliğin yerelleşmesi (*locality of change*), bir değişikliğin etkisinin yapıldığı parçanın içinde kalması ve başka parçalara yayılmamasıdır.

Aynı dosyadaki fonksiyonlar ortak değişkenleri ve yardımcı fonksiyonları doğrudan paylaşabilir. Bu paylaşım görünmez bağlantılar üretir: bir fonksiyon değiştirildiğinde, aynı ortak değişkene dayanan başka bir fonksiyon bozulur. Dosya bölündüğünde paylaşım ancak açık bir arayüz üzerinden yapılabilir ve bağlantı görünür hâle gelir. Parnas'ın (1972) *bilgi gizleme* (*information hiding*) ilkesi bu durumu tarif eder: her modül bir tasarım kararı içinde saklar ve o karar değiştiğinde yalnızca o modül değişir.

Üç mekanizmanın ilişkisi

Üç mekanizma aynı sınırın farklı sonuçlarıdır:

Mekanizma	Sınırın etkisi
Bilişsel yük sınırı	Dosya tek seferde akılda tutulabilir
Yüksek uyum	Dosya tek bir işe daralır
Değişikliğin yerleşmesi	Değişikliğin etkisi dosyanın içinde kalır

Sonuç

Satır sınırı ölçülmesi kolay bir büyüklüktür; uyum ve yerellik ise doğrudan ölçülemez. Sınırın pratik değeri buradan gelir: ölçülemeyen özelliklerin yerine ölçülebilen bir vekil koyar. Bir dosya sınırı aşıyorsa çoğu durumda ikinci bir işe başlamıştır ve sınır bu durumu erken bildirir.

Vekil ile asıl özellik aynı şey değildir ve çerçeve bu noktada bulanıklaşır. Tek iş yapan 250 satırlık bir dosya sınırı aşar ama sorun üretmez. Buna karşılık birbirine sıkı bağlı bir kodun yalnızca sınıra uymak için bölünmesi uyumu düşürür: tek bir işi anlamak için birden fazla dosya arasında gezmek gerekir ve bilişsel yük azalmak yerine artar. Sınırın ne zaman esnetileceği sorusu bu nedenle satır sayısına değil, dosyanın kaç iş yaptığına bakılarak cevaplanır.

Açık kalan soru, sınırın sayısal değerinin neye göre seçileceğidir. 200 sayısı deneyime dayanır; dile, ekibe ve aracın türüne göre değişmesi beklenir. Kod üreten dil modelleriyle çalışırken sınırın daha da önem kazandığı gözlenmektedir, ancak bu gözlemin sistematik bir ölçümü doğrulanmalıdır.

Aynı yapı yazılımın dışında da görülür. Simon'un (1962) neredeyse ayrıştırılabilir sistemler çerçevesi ile Baldwin ve Clark'ın (2000) modülerlik çalışması, parçaya üst sınır koyma ve sorunu parçanın içinde tutma fikrinin genel bir tasarım örüntüsü olduğunu gösterir.

Kaynaklar

- Baldwin, C. Y. ve Clark, K. B. (2000). *Design Rules, Volume 1: The Power of Modularity*. MIT Press.
- Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- Miller, G. A. (1956). The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information. *Psychological Review*, 63(2).
- Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12).
- Simon, H. A. (1962). The Architecture of Complexity. *Proceedings of the American Philosophical Society*, 106(6).
- Stevens, W. P., Myers, G. J. ve Constantine, L. L. (1974). Structured Design. *IBM Systems Journal*, 13(2).
- Sweller, J. (1988). Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science*, 12(2).